Session 09 - Object Iteration, JSON Stringify/Parse, Null Handling

Naveen AutomationLabs | 2026 Playwright Batch G1

1. Object Iteration - for...in Loop

The for...in loop is used to iterate over the keys (property names) of an object.

```
let user = {
name: 'Tom',
age: 30,
city: 'LA',
isActive: true
};

// Get all keys:
console.log(Object.keys(user)); // ['name', 'age', 'city', 'isActive']

// Iterate using for...in:
for (let key in user) {
console.log(key, ':', user[key]);
}
```

Iterating Nested Objects and Arrays

```
let user = {
name: 'Tom',
age: 30,
address: { flat: 101, zip: 545454 },
devices: ['iphone', 'mouse', 'keyboard']
};

for (let key in user) {
console.log(key, ':', user[key]);

if (key === 'address') {
for (let ele in user[key]) {
console.log(' ', ele, ':', user[key][ele]);
}
}
if (key === 'devices') {
for (let ele of user[key]) {
console.log(' ', ele);
}
}
}
```

Use for...in for objects (iterates keys). Use for...of for arrays (iterates values).

2. JSON.stringify - Serialization

JSON.stringify() converts a JavaScript object into a JSON string. This process is called serialization (or marshalling).

```
let user = { name: 'Tom', age: 30, city: 'LA', isActive: true };

let userJson = JSON.stringify(user, null, 2);
console.log(userJson);
// {
// "name": "Tom",
// "age": 30,
// "city": "LA",
// "isActive": true
// }
console.log(typeof userJson); // string
```

The second argument (null) is for a replacer function. The third argument (2) is for indentation/spacing.

3. JSON.parse - Deserialization

JSON.parse() converts a JSON string back into a JavaScript object. This is called deserialization (or unmarshalling).

```
let userObj = JSON.parse(userJson);
console.log(userObj);
console.log(typeof userObj); // object

console.log(user.name === userObj.name); // true
```

After stringify and parse, the new object is a completely separate copy -- not the same reference as the original.

4. Serialization with Nested Objects

```
let customer = {
name: 'Ravi',
age: 30,
address: { flat: 101, zip: 545454, lat: 10.34 },
devices: ['iphone', 'mouse', 'keyboard']
};

let customerJson = JSON.stringify(customer, null, 2);
console.log(customerJson);

let customerObj = JSON.parse(customerJson);
console.log(customer.devices.length === customerObj.devices.length); // true
console.log(customer.address.flat === customerObj.address.flat); // true
```

5. Null Object Reference and Optional Chaining

If you set an object to null and try to access a property, you get a TypeError.

```
let obj = { name: 'Tom', age: 20 };
obj = null;
// console.log(obj.name); // TypeError: Cannot read properties of null

// Safe way using optional chaining:
console.log(obj?.name); // undefined (no error)
console.log('hello world'); // code continues normally
```

The ?. operator checks if the value before it is null or undefined. If it is, it short-circuits and returns undefined instead of throwing an error.

6. When to Use What (API Automation Context)

- POST call: Convert JS object to JSON string using JSON.stringify() before sending to the server.
- API response: Convert JSON response to JS object using JSON.parse() to read data easily.
- Serialization = JS Object to JSON String
- Deserialization = JSON String to JS Object